

A type for utf-8 data

P0372R0

May 30, 2016

Michael Spencer <bigcheeseegs@gmail.com>

Davide C. C. Italiano <dccitaliano@gmail.com>

Audience: EWG

Introduction

We propose adding a new distinct type, `char8_t`, to represent UTF-8 encoded data.

Problem

The C++ standard currently confuses the native narrow encoding and UTF-8 encoding by representing them both as the type `char`. This makes it difficult to write portable programs that interact with both the native narrow encoding (most of the standard library) and UTF-8 (external libraries and some parts of the standard library).

Examples

- File names and paths
The standard library provides no overloads for functions which take paths or files names that are UTF-8 encoded.
- `codecvt`
The `codecvt` class treats `char` as UTF-8 and provides no way to perform conversions to or from the native narrow encoding.
- `filesystem::path`
The `u8string()` member function returns a `std::string` with UTF-8 encoding.

Solution

Add `char8_t` as a unique unsigned type with the same alignment, value representation and object representation as `unsigned char`. The intent is to allow explicit casting between `char*` and `char8_t*` when the encoding is known for interoperability.

Make `u8"..."` strictly a UTF-8 string literal with the type `const char8_t[]`.

Make `u8'.'` strictly a UTF-8 character literal with the type `char8_t`.

Make UTF-8 string literals convertible to narrow string literals.

Make UTF-8 character literals convertible to narrow character literals.

Examples

```
// In all cases the string is UTF-8.
const char8_t ua[] = u8""; // OK
const char    ca[] = u8""; // OK
const char8_t *u  = u8""; // OK
const char    *c  = u8""; // OK

const char    *e  = u; // ERROR - pointers to different types

void f(const char *);

f(u8 ""); // OK
f(u);     // ERROR - pointers to different types

void o(const char *);
void o(const char8_t*);

o(u8 ""); // OK - calls const char8_t*
o(u);     // OK - calls const char8_t*
o("");    // OK - calls const char*
o(c);     // OK - calls const char*
```

Where will it be used?

This proposal only adds the type and changes the behavior of `u8""` and `u8' '`. Future library proposals will use `char8_t` and friends to fill in basic unicode support for existing parts of the standard library such as.

- `u8string`
- `basic_fstream` filename parameter
- `basic_ios` unicode character types
- `filesystem::path` constructors from UTF-8

Why not a library implementation?

`char8_t` could be implemented as:

```
enum class char8_t : unsigned char {};
```

However this would require including a header to use, and would make the definitions of `u8""` and `u8' '` depend on the library. It would also have different conversion behavior from `char16_t` and `char32_t`.

Compatibility

This change loudly breaks any current usage of the identifier `char8_t`. All uses we found in open-source were typedefs to `char`, `unsigned char` or an equivalent type from `<cstdint.h>` and also used `char{16,32}_t` in the surrounding code.

This change also breaks code that relies on what `u8""` and `u8' '` type deduce to. We were not able to find any instances of this in open-source code.